



掌握Lambda表达式基础语法

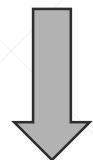
北京理工大学计算机学院
金旭亮

函数型接口的定义与实例化



```
@FunctionalInterface
public interface SimpleInterface {
    void doSomething();
}
```

定义



实例化

```
SimpleInterface obj = () -> System.out.println("this is a lambda example");
obj.doSomething();
```

UseSimpleInterface.java

如果有多句代码.....

当Lambda表达式包容多条语句时，可以使用“{”和“}”包围，如下所示：

```
SimpleInterface obj2 = () -> {  
    System.out.println("Hello");  
    System.out.println("world");  
};  
  
obj2.doSomething();
```

如果接口方法中定义有参数.....

带有参数的Lambda表达式，其参数被推断为int：

```
@FunctionalInterface
public interface InterfaceWithArgs {
    void doSomething(int value1, int value2);
}
```

```
public class UseInterfaceWithArgs {
    public static void main(String[] args) {
        InterfaceWithArgs obj = (v1, v2) -> {
            int result = v1 + v2;
            System.out.println("The result is " + result);
        };
        obj.doSomething(100, 200);
    }
}
```

UseInterfaceWithArgs.java



Lambda表达式中的参数名字无关紧要，也无需指明参数类型，因为编译器能依据接口定义，推断出Lambda表达式中方法参数的类型。这就是Lambda表达式的“**类型自动推断**”特性。

巩固练习



以下表格展示了一些典型的Lambda表达式的示例，请自行阅读理解它们的含义：

使用案例	Lambda 示例
布尔表达式	<code>(List<String> list) -> list.isEmpty()</code>
创建对象	<code>() -> new Apple(10)</code>
消费一个对象	<code>(Apple a) -> { System.out.println(a.getWeight()); }</code>
从一个对象中选择/抽取	<code>(String s) -> s.length()</code>
组合两个值	<code>(int a, int b) -> a * b</code>
比较两个对象	<code>(Apple a1, Apple a2) -> a1.getWeight().compareTo(a2.getWeight())</code>

这些Lambda表达式，都可以赋值给特定的函数型接口变量，你可以设计并写出这些接口吗？

从示例中进行自我总结.....



```
public class VisitVariableTest {  
    private final int counter = 100;  
  
    public static void main(String[] args) {  
        VisitVariableTest obj = new VisitVariableTest();  
        obj.test(info: "hello");  
    }  
  
    private void test(String info) {  
        String methodMsg = "test方法中定义的methodMsg变量";  
        Printer printer = localMsg -> {  
            System.out.println("Lambda可以访问自己定义的局部变量: msg=" + localMsg);  
            System.out.println("Lambda还可以访问它所在方法所定义的局部变量: methodMsg=" + methodMsg);  
            System.out.println("Lambda也能访问类的实例变量: counter=" + counter);  
            System.out.println("Lambda还可以访问方法的参数: info="+info);  
        };  
        printer.print("Hello,World.");  
    }  
}
```

```
@FunctionalInterface  
interface Printer {  
    void print(String msg);  
}
```

VisitVariableTest.java

仔细阅读上述代码，注意Lambda表达式中的代码能访问哪些变量，尝试总结出相应的规则。

闭包



在前面的例子中，我们可以看到，在Lambda表达式中的代码，是可以访问定义在表达式外部的变量的。



在程序运行时，**Lambda表达式 + 定义在表达式外部的变量**，构成一个“**闭包 (Closure)**”，你可以把闭包看成是Lambda表达式运行的总体“环境”。打个比方，人与地球也是一个“闭包”，因为人必须生活于地球所提供的“生态圈”中，两者是无法分离的。



“闭包”在许多面向对象的编程语言（比如JavaScript和C#）中都有，Java中的闭包特性与它们并没有什么本质上的区别。

动手动脑



以下代码无法通过编译，你知道原因吗？

```
public class Test {  
    public static void main(String[] args) {  
        String msg = "Hello";  
        Printer printer = msg -> System.out.println(msg);  
    }  
}  
@FunctionalInterface  
interface Printer {  
    void print(String msg);  
}
```



IntelliJ会突出显示有问题的代码，鼠标移上去，就能看到答案。

解惑



与函数不一样，**Lambda表达式并不定义一个新的变量作用域**，因此，示例代码实际是在同一个作用域内定义两个相同的变量名msg，自然无法通过编译了.....

```
public class Test {  
    public static void main(String[] args) {  
        String msg = "Hello";  
        Printer printer = msg -> System.out.println(msg);  
    }  
}
```



那怎么更正这一错误呢？这一任务留作练习。

“effectively final”的变量



Lambda表达式中的代码可以访问外部变量，但它要求这些变量必须是“effectively final”的。



所谓“effectively final”，主要针对局部变量而言的，它要不“**使用final定义**”，或虽然没有使用final定义，但“**只初始化了一次**”。

```
public Printer test() {  
    String msg="Hello";  
    Printer printer = info -> {  
        msg = msg.toUpperCase();  
        System.out.println(msg+info);  
    };  
    return printer;  
}
```

无法通过编译，因为msg变量被初始化了两次，这是不允许的。

知识测试



如果定义了这样的一个函数型接口：

```
@FunctionalInterface
interface Printer {
    void print(String msg);
}
```

那么请问，以下代码能通过编译吗？自己试一试，看看编译器告诉我们什么.....

```
public class Test {
    public static void main(String[] args) {
        String msg = "Hello";
        Printer printer = info -> {
            String msg = "Hi";
            System.out.println(msg+info);
        };
        printer.print("world");
    }
}
```

```
public class Test {
    public static void main(String[] args) {
        Printer printer = info -> {
            System.out.println(msg+info);
        };
        String msg="Hello";
        printer.print("world");
    }
}
```

小结



本讲主要介绍了Lambda表达式的基本编写方法，主要有以下几种情形：

(1) 单行表达式

(2) 使用{}包围的多行语句

后面还会介绍第三种方式，使用方法引用直接省略Lambda表达式的编写。



Lambda表达式的“闭包”特性比较重要，必须认真理解和把握。



请认真完成本讲所布置的练习。

动手动脑



由于Lambda表达式不创建新的变量作用域，因此，在Lambda表达式中出现的`this`就有着微妙的含义。

请仔细阅读并运行`ThisTest.java`示例，自己分析和体会示例代码中`this`到底引用哪个对象，然后自己总结相应规则。

编程技能训练



JDK中定义了一个`Comparator<E>`接口，可用于定义两个对象相互比较大小的“比较规则”。`Collections.sort()`方法的一个重载形式可以使用一个`Comparator<E>`对象来对一个对象集合排序。

现在有一个包容了多个英文单词的数组，请使用`Comparator<E>`接口和`Collections.sort()`方法对这个数组中的单词按照字典顺序（不区分大小写）进行排序。要求使用Lambda表达式实现。

参考解答：UseComparator.java

下一讲



方法引用